

Attested Compute in hanzod: Host-Asserted Evidence over a Monitor’s Measurement, and the Gap to Hardware Attestation

Hanzo AI Research*
Hanzo Industries Inc (Techstars ’17)
Los Angeles, California
<https://hanzo.ai>

Abstract

hanzod runs work for other parties on hardware those parties cannot inspect. If that work is to be paid for on a settlement chain, the chain needs some statement about what ran, and needs to know what the statement is worth. This paper specifies the statement **hanzod** makes today and states its limits. The measurement itself is not **hanzod**’s: the **hanzo-vm** monitor folds the kernel, the kernel command line it builds, the root image and the machine shape into an extend-only SHA-384 register, under the same extend rule Intel TDX applies to a runtime measurement register, and emits a document whose **bind** field is the 64 bytes an SEV-SNP report or a TDX quote carries as caller data. **hanzod** takes that document, signs the **bind** with the node’s ed25519 identity key, and publishes a record. We define *evidence* as a signed statement about a bind and distinguish two roots for it: *host-asserted* evidence, signed by the node’s own key, which is implemented; and *hardware attestation*, rooted in a vendor certificate chain, which is not, and which cannot be produced by **hanzod** at all because the report is requested from inside a guest and **hanzod** runs on the host. Host-asserted evidence establishes that a particular node made a particular claim and that the claim has not been altered since; it does not establish that the node fed those images to the monitor. We report an implementation of 892 lines of Rust with 46 tests, wired into **hanzod** as **hanzod machine**, and an end-to-end run on Apple M1 Max that measures a real microVM through **hanzo-vm**, signs the measurement with the node’s identity key, boots the machine, publishes the record, and reports how the machine ended. We verify one such record with OpenSSL and a re-fold of the register, using no Hanzo code. We then state what remains: a guest-side attester that binds the same 64 bytes into a hardware report, a workload register that covers the guest command, and a registry that outlives the process.

1 Problem

A node sells compute. A buyer pays for it. Between the two sits a question the buyer cannot answer by looking: *what actually ran?* The buyer sees a result, or an endpoint, and has no view of the machine that produced it.

Three claims are commonly conflated, and it is worth separating them before anything else:

- **Integrity of the inputs.** The images fed to the hypervisor were the images they were said to be.

*Corresponding author: research@hanzo.ai

- **Fidelity of execution.** Those images, and only those, determined what ran.
- **Quality of the result.** What ran produced a correct answer.

They are different properties with different mechanisms. A digest over images addresses the first. A hardware root of trust addresses the second. Neither addresses the third; that is the domain of verification schemes over the computation itself, and is out of scope here. This paper is about the first, and about being exact regarding how far short of the second it falls.

2 Current behavior

This section reports what exists, with locations, so that later claims can be checked against code rather than against prose.

2.1 The monitor, and where the measurement lives

`hanzo-vm` (repository `hanzoai/vm`, crate `hanzo-vm` 2.0.1) is the virtual machine monitor. It uses `Virtualization.framework` on macOS and KVM or cloud-hypervisor on Linux. Its operating-system assets live in `$HANZO_VM_HOME`, defaulting to `~/.hanzo/vm`: a kernel image (`Image`), a base root filesystem (`rootfs.ext4`), an optional `initramfs`, a `VERSION` file, and a `checkpoints/` directory holding saved disk states.

The measurement is the monitor’s, in crate `vm-measure`. It is an ordered log of events folded into a single register:

$$e_i = \text{SHA-384}(\text{name}_i \parallel 0x00 \parallel \text{SHA-384}(\text{content}_i)), \quad R_0 = 0^{384}, \quad R_{i+1} = \text{SHA-384}(R_i \parallel e_i).$$

This is the extend rule Intel TDX applies to a runtime measurement register, so a register built this way reproduces an RTMR that started at zero and was extended with the same events in the same order. SHA-384 is the digest both TDX registers and the AMD SEV-SNP launch measurement use.

Two logs are kept separately. *Launch* is what the hypervisor was handed: the kernel image, the command line the monitor builds, the `initramfs` when there is one, the source root image, and the machine shape as the text `cpus=c memory=mMB disk=sMB`. *Workload* is what the started system was told to run, and is extended by whoever deploys into the machine. The two are separate because on TDX they map onto separate registers, and folding them would lose which half a verifier is entitled to predict.

Definition 1 (Bind). The *bind* of a measurement is $\text{SHA-512}(R_{\text{launch}} \parallel R_{\text{workload}})$, 64 bytes. SHA-512 because the caller field of an SEV-SNP report and of a TDX quote is 64 bytes wide, and a report that pads is a report where the padding is unaccounted for.

Two properties of the fold matter downstream. An event’s name is inside its own digest, so an event cannot be replayed under another name, and the chain commits to order. Only content is hashed: the path a file was read from is recorded for the reader and never enters a digest, so the same kernel and the same root image measure identically on every machine that has them.

Published boot medians for this monitor, measured across three hosts, are 0.30 s (Apple M4 Max, `Virtualization.framework`), 0.27 s (Ryzen AI Max+ 395, cloud-hypervisor) and 0.30 s (GB10, KVM). Those are boot alone. The wall times in §7 are whole invocations — disk clone, boot, the guest’s command, and teardown — and are the larger number for that reason.

2.2 The Kubernetes path

`hanzo up` (repository `hanzoai/cli`) boots `k3s` inside one such microVM and writes a kubeconfig. It supervises `hanzo-vm run --stdio` as a child and speaks JSON-RPC 2.0 over that child's stdio. First boot creates a disk checkpoint named `k3s`; every later boot starts from it. This is the path by which an OCI image — for the cloud control plane, `ghcr.io/hanzoai/cloud` — reaches a microVM. It is also the existing consumer of the monitor's measurement: it receives the launch log over stdio and extends the workload register with the manifest it deploys.

2.3 Where machines are recorded today

Two durable surfaces exist, and neither holds a measurement.

`/v1/machines`, served by `hanzoai/visor`, is the org's machine inventory: owner, name, provider, region, image, status, addresses, tags. It records *that* a machine exists and who owns it.

`/v1/node`, specified in HIP-1325 and implemented in `hanzoai/cloud`, is an org's own machines connected by an agent that dials in and holds a socket. The specification is explicit that it keeps no durable store: it is a presence map. It records *that* a machine is currently reachable.

Neither surface has a field for what a machine booted from, and neither carries a signature. That absence is the gap this work begins to close.

2.4 Prior code in the node repository

`hanzoai/node` already contains a security module (`hanzo-bin/hanzo-node/src/security/`, six files) implementing a TEE attestation framework with SEV-SNP, TDX, SGX and NVIDIA confidential-computing detection, keyed to a `PrivacyTier` type from a `hanzo-kbs` crate. It is not compiled: the crate it depends on is excluded from the workspace as incomplete, and the module is not declared in either `main.rs` or `lib.rs`. A `compute_dex` module in the same tree binds Solidity contracts that already carry a `bytes32 attestation` field on orders; it is likewise undeclared and depends on a crate not listed in the manifest.

We report this rather than extend it. The present work does not replace that framework's ambition; it establishes the layer beneath it — a statement that is defined, produced and tested — so that a hardware attester, when written, has something specific to bind to.

3 What hanzod adds, and what it deliberately does not

The monitor answers *what was launched*. It does not answer *on whose word, for whom, or where the answer is recorded*. Those are `hanzod`'s, and they are the whole of what this work adds.

`hanzod` therefore holds no measurement code. It builds one description of a machine and derives both monitor invocations from it, so what is measured and what boots cannot drift:

```
hanzo-vm measure --cpus 2 --memory 2048 --disk-size 8192 --from k3s
hanzo-vm run --cpus 2 --memory 2048 --disk-size 8192 --from k3s \
    --allow-net -p 6443:6443 -- k3s server
```

The flags `run` carries and `measure` does not — port forwards and network egress — are exactly the ones that do not enter the launch register. They change what the machine can reach, not what was launched. A record says so rather than implying they were measured (§8).

The argument for not computing a second measurement in `hanzod` is not economy of code. A second fold would be a second answer to a question that has one, and the two would diverge the first time the monitor changed what it hands the hypervisor — silently, because a measurement

that is merely wrong still verifies. It would also be the wrong number: the monitor’s register is SHA-384 under the TDX extend rule and its bind is 64 bytes wide, so it can be placed in a hardware report’s caller field unchanged, and a register computed here under any other rule could not.

4 Evidence and its trust base

A bind is a number anyone holding the same images can compute. It becomes useful only when some party stands behind it.

Definition 2 (Evidence). *Evidence* is a tuple $(B, \kappa, p, \sigma, t)$: a bind, a kind κ , a public key p , a signature σ , and a timestamp t .

The signed preimage is

$$\text{hanzo.evidence.v1} \parallel \kappa \parallel B \parallel t$$

with κ a single byte and t a big-endian 64-bit Unix second. Including κ means a signature made as one kind cannot be presented as another. Including t means a statement cannot be replayed later under a fresher date: re-dating it invalidates σ . Evidence is therefore dated to the second it publishes, and not more finely — anything below the resolution of t would sit outside σ and could be altered without breaking it.

4.1 Two kinds, and the difference between them

host The node signs B with its own ed25519 identity key — the same key it is known by on the network, read from the same `.secret` file the daemon uses. *Implemented.*

snp, tdx A report produced by AMD SEV-SNP or Intel TDX over the same 64 bytes, rooted in a vendor certificate chain rather than in the node. *Not produced by hanzod*, and not for want of finishing: those reports are requested over `/dev/sev-guest` and `/dev/tdx-guest`, which are guest devices, and **hanzod** runs on the host. The monitor already implements the two request ABIs and reports which platform it found; on every machine this has run on, the answer is **none**. **hanzod** carries that answer and does not claim it.

	host	snp/tdx
Statement is attributable to a named key	yes	yes
Statement cannot be altered in transit	yes	yes
Statement cannot be replayed under a new date	yes	yes
Images were the ones named	<i>if the node is honest</i>	yes
Node cannot forge the statement	no	yes
Available in this implementation	yes	no

Table 1: What each kind of evidence establishes. The fourth and fifth rows are the entire difference, and they are the rows a settlement chain cares about.

4.2 Adversaries

A network adversary between the node and whoever reads its records can modify what is in flight. Host-asserted evidence defeats this: the record is self-contained and signed, so a reader re-folds the monitor’s log, compares the result to the bind inside the signature, and checks the signature — without trusting the transport.

A confused or buggy node that boots something other than what it intended. The measurement catches this to the extent that the divergence is in the launch inputs: a different checkpoint, a different kernel, a different shape all change the register.

A dishonest node operator who wants payment for compute not delivered, or delivered on a cheaper image. Host-asserted evidence does *not* defeat this. The operator measures the images they wish to claim, signs the bind with their own key, and boots whatever they like. The signature verifies. This is not a defect in the implementation; it is the definition of a host-asserted root, and it is why hardware attestation is the next piece of work and not an optional refinement.

Pre-boot measurement on the host is therefore a statement about intent, made by the party whose intent is in question. It is worth exactly the reputation of the signer, and a settlement chain reading these records today should price them accordingly.

5 Registration

Definition 3 (Record). A *record* is $(\text{node}, \text{vm}, M, E, t)$: the node’s identity, the machine’s identifier, the monitor’s measurement document, the evidence, and the time of publication.

The record carries the monitor’s whole document rather than only the bind, so a reader who never met the node can re-fold the log, recompute the bind, and compare it to the number inside E . We call a record satisfying $B_M = B_E$ *coherent*. Coherence is cheap, local, and independent of the signature: a record can be internally inconsistent while carrying a perfectly valid signature over some other bind, and the check that catches that is comparison, not cryptography.

Verifying a signature and deciding whether its signer deserves belief are separate questions, and the implementation separates them. The **Verify** trait answers the first. The second belongs to whoever reads the registry — a scheduler, an operator, a chain — and is not represented in this crate at all.

The implemented registry is the node’s own view: records are added when a machine starts, removed when it stops, and gone when the process is. This matches the durability of `/v1/node` (§2.3), which by specification holds no durable store. A registry that outlives the process is a second implementation of the same four-method trait and requires no change to anything above it.

6 Implementation

The crate is `hanzo-attest`, at `hanzo-libs/hanzo-attest` in `hanzoai/node`, 892 lines of Rust across seven modules.

Three traits keep the parts separable: **Monitor** (measure, start, wait, stop, running), **Attest/Verify**, and **Registry** (put, get, remove, list). The monitor knows nothing about signing, the key knows nothing about machines, and the registry knows nothing about either. **Node** is

Module	Holds	Lines
monitor	the two argv, holding the child, waiting on it	280
evidence	kinds, signing preimage, attester, verifier	196
measure	the monitor’s document, as hanzod holds it	116
registry	records, coherence, the node’s own view	93
node	the composition	91
lib	the crate root and hex encoding	81
error	every way the crate fails	35

Table 2: Modules. Line counts are of implementation only: everything above each file’s test module, comments and blank lines included.

the single place they meet, which is what makes the ordering a property of the type rather than a convention: a machine is never registered before it is measured, and never left running unregistered.

Monitor::stop is idempotent. A machine that has already exited is the state being asked for, not a failure, so a reconciler never has to distinguish the two cases.

Monitor::wait exists because starting a machine is spawning the monitor, and a monitor that refuses the configuration refuses it *after* the spawn has already succeeded. Without waiting, **hanzod** would print a record for a machine that never came up and exit successfully. **Node::serve** waits, withdraws the record whichever way the machine goes, and returns how it ended; **hanzod** exits with that code, so a machine that failed is a command that failed.

6.1 The contract with the monitor

Nothing here reimplements what **hanzo-vm** already does. The crate builds the two argv above, runs **measure** to completion, parses the document, and spawns **run**, holding the child. The guest’s console is inherited, so what the machine prints lands in **hanzod**’s log. The JSON-RPC supervision path (**--stdio**) belongs to **hanzo up** and is not duplicated.

The parse is deliberately narrow: three fields — the algorithm, the bind, and what the monitor said about the platform — and the rest of the document carried without rewriting. A document naming any algorithm but **sha384**, or a bind that is not 64 bytes, is refused. A monitor too old to have a **measure** subcommand fails the same way, and the failure is a refusal to publish rather than a fallback to some number **hanzod** computed itself.

6.2 The command

hanzod gains one subcommand, on the same identity the daemon uses:

```
hanzod machine measure [shape] [-- argv] # what would boot, and the evidence
hanzod machine run [shape] [-- argv] # boot it, publish, hold it
```

The two differ in exactly one thing: whether a machine actually starts. Both ask the monitor the same question about the same machine and sign the same answer with the same key, so **measure** is how an operator inspects what a **run** will publish before publishing it. Bare **hanzod** is unchanged.

Identity resolution was, before this change, inline in the daemon’s startup path. It is now one function, **runner::identity**, called by both, so the daemon and the subcommand cannot disagree about who the node is.

7 Evaluation

Tests. 46 tests pass: 41 in **hanzo-attest**, one documentation test, and four over the command’s argument parsing in **hanzod**. The measure tests assert that a document yields the bytes a report would carry, that the monitor’s JSON is carried unrewritten, that an unknown fold and a bind of the wrong width are refused, and that a hardware report is noticed without being claimed. The evidence tests assert that a signature fails after the bind, the date, the signer or the kind is altered. The monitor tests assert both `argv`, that the flags **measure** omits are the ones that are not measured, that a monitor which will not measure does not boot, that a machine which never came up is not a success, that a machine killed outright reports 128 plus its signal, and the child bookkeeping — using a shell script that prints a document and does as it is told, in place of a hypervisor.

End-to-end. On Apple M1 Max, macOS 26.6.2, with **hanzo-vm** 2.0.1 and its 2.0.0 OS assets:

```
$ hanzod machine run --cpus 2 --memory 2048 --disk-size 8192 \  
-- /bin/sh -c "uname -srm; nproc; free -m"
```

writes one record to standard output and the guest’s console to standard error, where the guest reports `Linux 6.12.17 aarch64`, two processors and 1998 MB of memory — the shape that was asked for and measured. The launch log carries five events (`kernel`, `cmdline`, `initrd`, `root`, `shape`); the workload register is zero (§8); `platform` is `none`; the evidence kind is `host`.

	hanzo-vm alone	under hanzod
measure, images already digested	0.05–0.06 s	0.05–0.13 s
measure, first time (4 GiB root image)	44.2 s	—
run a trivial guest command	1.10–2.31 s	1.07–1.09 s

Table 3: Three runs each, same host. **hanzod** adds a process invocation, a JSON parse and one ed25519 signature to a boot, and the columns are within each other’s spread. The cold measurement is the monitor reading the root image; its digest cache is keyed to the file’s identity and is host-local, so it saves the 44 seconds and establishes nothing.

The record checks out under someone else’s tools. Taking that record and using no Hanzo code: re-folding the launch and workload logs with SHA-384 reproduces both register values; SHA-512 over the two registers reproduces the `bind`; the `bind` equals the one inside the evidence; and OpenSSL verifies the ed25519 seal over `hanzo.evidence.v1 || 0x01 || bind || t` (90 bytes) under the signer named in the record. This is the property the record is for: a reader who never met the node establishes what the node said without running anything the node wrote.

What the failure paths do. A monitor too old to have a **measure** subcommand — the released **hanzo-vm** 2.0.0 — is refused: **hanzod** prints the monitor’s own complaint, publishes nothing and exits 1. A monitor that measures but cannot boot — on macOS, one lacking the `com.apple.security.virtualization` entitlement — exposes the failure the spawn cannot see: the record is published, the machine never comes up, and `Node::serve` withdraws the record and exits 1. Changing the shape changes the `bind`. A guest command exiting 42 makes **hanzod** exit 42.

8 Limitations

1. The measurement is taken before the boot, on the host, over images on the host. It binds the inputs. It does not witness the execution, and a dishonest host produces evidence that verifies (§4.2).
2. **hanzod** cannot produce **snp** or **tdx** evidence at all: a report is requested from inside the guest and **hanzod** runs on the host. The monitor's document carries which platform was found; on every machine this has run on it is **none**.
3. The workload register is empty for machines **hanzod** starts. **hanzo-vm measure** takes no trailing command, so the argv the guest runs is not folded into any register. It is recorded in the run invocation and in **hanzod**'s own log, and it is **hanzod**'s claim, not a measured input. Closing this is a change to the monitor, not to **hanzod** (§9).
4. Port forwards and network egress do not enter the launch register. Two machines that differ only in what they can reach measure identically. This is a property of the monitor's launch log, reported here rather than worked around.
5. Measuring and booting are two invocations of the monitor, so a host that changed an image between them would measure one thing and boot another. Under a host-asserted root this costs nothing — a host that would do it can already sign for images it never booted — but it is a real window, and it is the monitor's to close by emitting the document for the launch it performs.
6. The root image measured is the source the monitor clones. The guest writes to the copy; nothing here measures the machine's state after it starts, and a long-running machine's disk diverges from its measurement immediately and by design.
7. The registry does not outlive the process. A record is the node's current claim, not a history.
8. Machine shape is what **hanzod** asked the monitor for. It is honest about intent, but it is not a measurement of delivered performance: two hosts granting the same shape need not deliver the same throughput.
9. A hard kill of **hanzod** leaves a running machine to the monitor. **machine run** withdraws its own record on exit; it cannot withdraw one after **SIGKILL**.

9 Next step

The following are proposals, not implemented behavior.

A guest-side attester. On a host with SEV-SNP or TDX, an agent inside the guest asks its firmware for a report over the bind and returns it. The evidence structure needs no change and neither does the number: the bind is already 64 bytes and already the value the caller field takes. The kind becomes **snp** or **tdx**, the signer becomes the vendor chain, and the verifier gains a certificate path. This is the change that moves rows four and five of Table 1. A verifier then owes one further check the log alone cannot supply: the platform's own launch measurement, carried in the same report, against an expected value.

A workload register that covers the command. `hanzo-vm measure` accepting the trailing command that `hanzo-vm run` already takes, and extending the workload log with it, would put the guest's argv inside the number `hanzod` signs. It is a change in the monitor because the fold is the monitor's; making it in `hanzod` would reintroduce exactly the second answer §3 argues against.

A durable registry. A second `Registry` implementation writing to `/v1/node`, or to a chain. The record is already self-contained and signed, so the transport need not be trusted; what a durable registry adds is history, and history is what a settlement window is computed over.

The settlement read. A chain that prices these records must decide what a record is worth, which is a policy question this crate deliberately does not answer. The minimum it needs is present: a coherence check that costs one comparison, a re-fold that costs one pass over the log, a signature check that costs one verification, and a kind field that tells it which of the two trust bases it is being offered.